

An Introduction to Algorithmic Trading and Backtesting

Here we create a python programme which backtests algorithmic trading strategies using the MACD, the RSI, and Bollinger Bands. The aim here is to understand how algorithmic trading is done using Python and how backtesting can be carried out so as to be beneficial to traders when deciding whether or not a particular strategy could perform well. We use various online sources to research the aforementioned tools, how they are calculated, how they are interpreted, and how they are incorporated into algorithmic trading strategies. We then build on the work by ATJ Traders [1] to simulate and backtest various strategies on historical price data, computing some basic metrics to analyse the performance of the backtest.

The MACD

Sources: [4], [5], [7]

The MACD indicator utilises what are known as exponential moving averages. Moving averages help us gauge the movement of the price process of an asset over a given time period. The n -period *simple moving average* (SMA) is the average of the last n periods for which we have price data. *Exponential moving averages* place a greater weighting to more recent price changes than simple moving averages. Exponential moving averages (EMAs) can be defined recursively and depend on the period specified. The n -period EMA process $(E_k^{(n)})_{k \geq 1}$ for a price process $(X_k)_{k \geq 1}$ - where X_k is the price of an asset on the k -th day and $E_k^{(n)}$ is the n -period EMA on the k -th day - is defined by the following relation:

$$E_k^{(n)} := \frac{2}{n+1} (X_k - E_{k-1}^{(n-1)}) + E_{k-1}^{(n-1)}.$$

EMAs are backward looking in the sense that they identify trends in the history of the asset price. The size of the period, n , also impacts the sensitivity of the EMA to changes in price.

The MACD (Moving Average Convergence Divergence) is the difference between two EMAs. Typically one uses the 26-day and 12-day EMAs of the asset price process. Then if $(M_k)_{k \geq 1}$ is the MACD process for $(X_k)_{k \geq 1}$, we have:

$$M_k := EMA_k^{(12)} - EMA_k^{(26)}.$$

Traders use the MACD to help identify changes in price momentum. The fact that EMAs place greater weighting on more recent price changes is significant here. The 12-day EMA is more sensitive to changes in price than the 26-day, and therefore a sudden rise in price will be felt more greatly by the former than the latter, hence the MACD will rise. A sudden fall in price will again be felt more greatly by the short term EMA than the longer; both EMAs will still decrease, but the varying sensitivity to this price change will mean that the MACD will decrease. Increases and decreases in the MACD act to signal changes in market sentiment, which can be used as evidence for exit and entry points for trades.

Associated to the MACD are the following:

- The Signal Line, typically defined as the 9-day EMA of the MACD process itself.
- The Histogram, which tracks the difference between the MACD and its signal line.

The most common trading rule with the MACD is the following:

- If the MACD line rises above the signal line, buy.
- If the MACD line falls below the signal line, sell.

This trading logic is not perfect. The MACD line may cross the signal line many times in a short time frame which suggest the stock price has no real momentum. Often traders use the MACD in conjunction with other indicators to get a clearer picture, hence what we are doing here. Traders may also increase the time scale of the MACD in relation to the price data available to them. For example, if the trader has daily price data, they may use a weekly MACD. This approach seeks to reduce the chance of a false signal when a crossover occurs.

We can also grasp some idea of the trend of the stock price via the MACD through instances when the MACD line crosses above or below 0:

- If the MACD line crosses below 0, there is evidence of a downwards trend.
- If the MACD line crosses above 0, there is evidence of an upwards trend.

The MACD can also be used to spot reversals and continuations of trends. We read in [7] that one can deduce regular and hidden bullish divergence, as well as regular and hidden bearish divergence. We do not consider divergence in this project, mostly because we have not yet had time to research how one can quantify divergence. In this project we focus mostly on establishing trends using the zero-line crossovers as well as leveraging the parameter settings for the constituent EMAs of the MACD for finding evidence of shorter and longer term trend directions.

The MACD Histogram

The MACD histogram is computed as the difference between the MACD line and the corresponding signal line. If positive, we consider there to be upwards price momentum, since the MACD is performing better than its 9-day EMA. Analogously if the histogram is negative, there may be downwards price momentum. The greater the difference, the greater the length of the histogram bars, and the greater the strength of the price momentum. We say there is *bullish momentum* if the histogram is positive and increasing. We say there is *bearish momentum* if the histogram is negative and decreasing. As the histogram bars shrink, we say there is weakening momentum.

Bollinger Bands

Sources: [2], [3], [9]

Let $(X_{t-i})_{i \in \{0, \dots, (m-1)\}}$ be the price of a risky security over the past $m \geq 1$ trading days. Set $\bar{X}_t^{(m)} := \frac{1}{m} \sum_{i=0}^{m-1} X_{t-i}$ be the m-day moving average on day t . Let the **linearly weighted moving average** and **rolling m-day standard deviation** be given respectively by:

$$\hat{X}_t^{(m)} := \frac{2}{m(m-1)} \sum_{i=0}^{m-1} (m-i)X_{t-i}, \quad \sigma_t^{(m)} := \left(\frac{1}{m-1} \sum_{i=0}^{m-1} (X_{t-i} - \bar{X}_t^{(m)})^2 \right)^{\frac{1}{2}}.$$

Then the upper Bollinger band, r^+ is defined as:

$$r_t^+ := \hat{X}_t^{(m)} + 2\sigma_t^{(m)},$$

and the lower Bollinger band, r_t^- as:

$$r_t^- := \hat{X}_t^{(m)} - 2\sigma_t^{(m)}.$$

An example of a strategy using Bollinger bands is known as the breakout strategy: to sell when X_{t+1} exceeds r_t^+ and buy when X_{t+1} falls below r_t^- ; the former case is where the Bollinger band represents "resistance", the latter representing "support".

Bollinger bands widen when there has been a price increase, and narrow when there is a price decrease, as we can see from the definitions of r_t^+ and r_t^- . The number of volatilities (in our case, 2) chosen for the distance of the bands from the simple moving average dictate how strong we want our trends to be before we make any trends. The lower the number, and thus the narrower the bands, the weaker our trends may be when the price process crosses either band. However, narrower bands mean more crosses and thus more trades placed.

Note that we could replace the Bollinger bands with the running maximum of the stock price over the same time period.

Another example of a strategy using Bollinger bands is known as a reversal strategy, in which we buy when the price exceeds the lower band after previously crossing below it, and sell when the price falls below the upper band after previously crossing above it.

Bollinger bands do have limitations, for example:

1. They are backward looking.
2. They do not perform as well in sideways markets.
3. When market volatility is low, we may get false signals.

The RSI

Sources: [6], [8]

The Relative Strength Index (RSI) is the third and final indicator we will consider in this project. The RSI tells us about how fast and by how much price changes can occur. It is an oscillator that ranges from 0 to 100 and contains two thresholds: 'Overbought' and 'Oversold'. Typically the overbought threshold is set at 70 and the oversold at 30, although these can be adjusted. For example, if a stock is trending upwards, we may set the thresholds at 80 and 40.

To compute the RSI, we use the code given in [1]. An explanation as to what the code is doing is given below:

If $(X_n)_{n \geq 1}$ is the daily asset price process and $(R_n)_{n \geq 1}$ is the corresponding RSI process, the RSI is computed as follows:

$$R_n := 100 - \frac{100}{1 + RS_n}, \quad n \geq 1.$$

Where RS_n is the relative strength of the asset price computed as the ratio between the EMA gain and EMA loss.

If $G_n := (X_n - X_{n-1})_+$ and $L_n := (X_n - X_{n-1})_-$ for $n \geq 1$ are the respective gains and losses from days $n - 1$ to n , then:

$$RS_n := \frac{EMA \text{ of } \{G_n : n = 1, \dots, k\}}{EMA \text{ of } \{L_n : n = 1, \dots, k\}}.$$

k is typically set to 14.

As pointed out in [8], a common trading logic with the RSI is the following:

- Once the RSI rises above the 'Overbought' threshold, sell.
- Once the RSI falls below the 'Oversold' threshold, buy.

However, a better strategy is the following reversal strategy:

- Once the RSI rises above the 'Overbought' threshold, sell once it falls below the threshold again.
- Once the RSI falls below the 'Oversold' threshold, wait for it to rise above the threshold again, then buy.

Traders can also identify trends with the RSI. Staying above 50 typically means the market is trending upwards whilst being consistently below 50 suggests the opposite. Often multiple timeframes can be leveraged here. Indeed, we learn from [8] that after identifying the long-term trend of the stock price, we can 'filter RSI buy and sell signals to match the long-term direction'. In this project we use the MACD to identify these long term upwards and downwards trends. We use the aforesaid RSI strategy for the buy/sell signals.

In this project, we will define functions which compute the above indicators for given historical price data. From these functions we can identify buy/sell signals based on given trading logic. Traders, when building a trading strategy, will test their trading logic on historical data to see if there is any merit in the strategy. This is known as **backtesting**. Backtesting forms part of the process of accumulating evidence that a trading strategy is likely to be profitable. In this project we give examples of backtests of given trading strategies. Below we outline how we measure the performance of the backtest.

Evaluating Backtest Performance:

Examples of metrics we use to measure the performance of our backtest are the following, taken from [11]:

1. Return on equity.
2. Net Profit/Loss
3. Historical volatility of returns.
4. Sharpe Ratio.
5. Beta

We specify a starting balance for us to trade with, we backtest our strategy on historical data, keeping track of our balance throughout the time horizon. From this we can compute the daily returns of the strategy, from which we can compute the volatility of returns, the Sharpe ratio, and the beta. For the Sharpe ratio and beta, we of course need a risk-free asset and a benchmark asset to measure the performance of the

trading strategy against. Typically, we use some sort of fixed income asset such as US Tbills as the risk-free asset. In this project, for the benchmark asset, we use the strategy of simply buying and holding the asset over the time horizon of the trading strategy.

We define the **Sharpe ratio** as the risk-adjusted return of our backtest. For a given risk-free rate - e.g. 10Yr US Tbill Yields - we compare the return on our backtest to that of investing in the risk-free asset over the same time horizon, standardising the difference to account for the difference in volatility of the two possible strategies.

If $(r_i)_{i=1}^N$ are the daily log returns on the risk-free asset over the time horizon and $(R_i)_{i=1}^N$ are the corresponding daily log returns on the backtest. As per chapter 3.1 of [2], we define the excess return, to be the difference between the log returns on the book and the log-returns on the risk-free asset, $y_i := R_i - r_i$ for $i \in \{1, \dots, N\}$. The Sharpe ratio, as in [12], is defined to be:

$$Sharpe = \frac{\bar{y}}{\sqrt{\frac{1}{N-1} \sum_{i=1}^N (y_i - \bar{y})^2}}. \quad (1)$$

Where $\bar{y} = \frac{1}{N} \sum_{i=1}^N y_i$ is the mean excess return over the period.

We define the beta (β), as per [13], of our trading strategy to be:

$$\beta = \frac{Cov(R_{\text{strat}}, R_{\text{Benchmark}})}{Var(R_{\text{Benchmark}})}. \quad (2)$$

Where R_{strat} is the vector of daily log returns of the strategy over the time horizon and $R_{\text{Benchmark}}$ is the vector of daily log returns of the benchmark over the time horizon.

The beta of the strategy indicates to us how sensitive the strategy is relative to the benchmark. By the benchmark we could mean an index such as the S&P500, or in our case we could take the underlying asset of our trading strategy as our benchmark, and compare the returns of the strategy to just simply buying the asset and holding. $\beta > 1$ suggests the strategy is more sensitive, for example $\beta = 1.5$ means that for every movement in the underlying asset, the strategy moves by a further 50%.

Backtesting Strategies

We now specify some basic trading strategies for us to backtest. The programmes themselves which carryout the backtests can be found in `AT.py` , and the functions which compute the technical indicators for a given dataset of historical data can be found in `Indicators.py` .

The reader should note that we have not put all too much thought into the strategies themselves, and the below code is simply an illustration of how a backtest might look for a given strategy. The data we have obtained is chosen purely on what we could find online, and the stop-loss, take-profit, and volume parameters for each trade are set purely as example parameters. As such, most strategies do not perform very well and anyone who invests real money into trading with these strategies should not come crying to me if they lose everything within the first week. You have been warned!

In this project, stop-loss and take-profit levels are set to 5% in their respective directions, and each trade is 100 shares. The starting balance for all algorithms will be 100,000 in whatever currency is required.

Bollinger Reversal

Here we backtest the Bollinger band reversal strategy mentioned above. We decide on `m=14` and for the bands to be `2` standard deviations from the linearly weighted moving average. We backtest the strategy on equity data from online sources listed in the bibliography.

Importing the Data

```
In [44]: # import the necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import datetime as dt
import Indicators as Ind
import AT as AT
pd.set_option('future.no_silent_downcasting', True)
```

```
In [19]: # define a function which formats our data - inputted as a .csv file - into the format which suits our programme:
def format(data_title: str, date_title: str, closing_price_title: str):
    """
    data_title = title of the .csv file containing the data
    date_title = title of the column under which the dates of the historical data are stored
    closing_price_title = title of the column under which the closing price data is stored
```

We wish for all of the data to have two columns: 'Date' and 'Close', for the dates to be dt.Datetimes and for the closing prices to be floats.

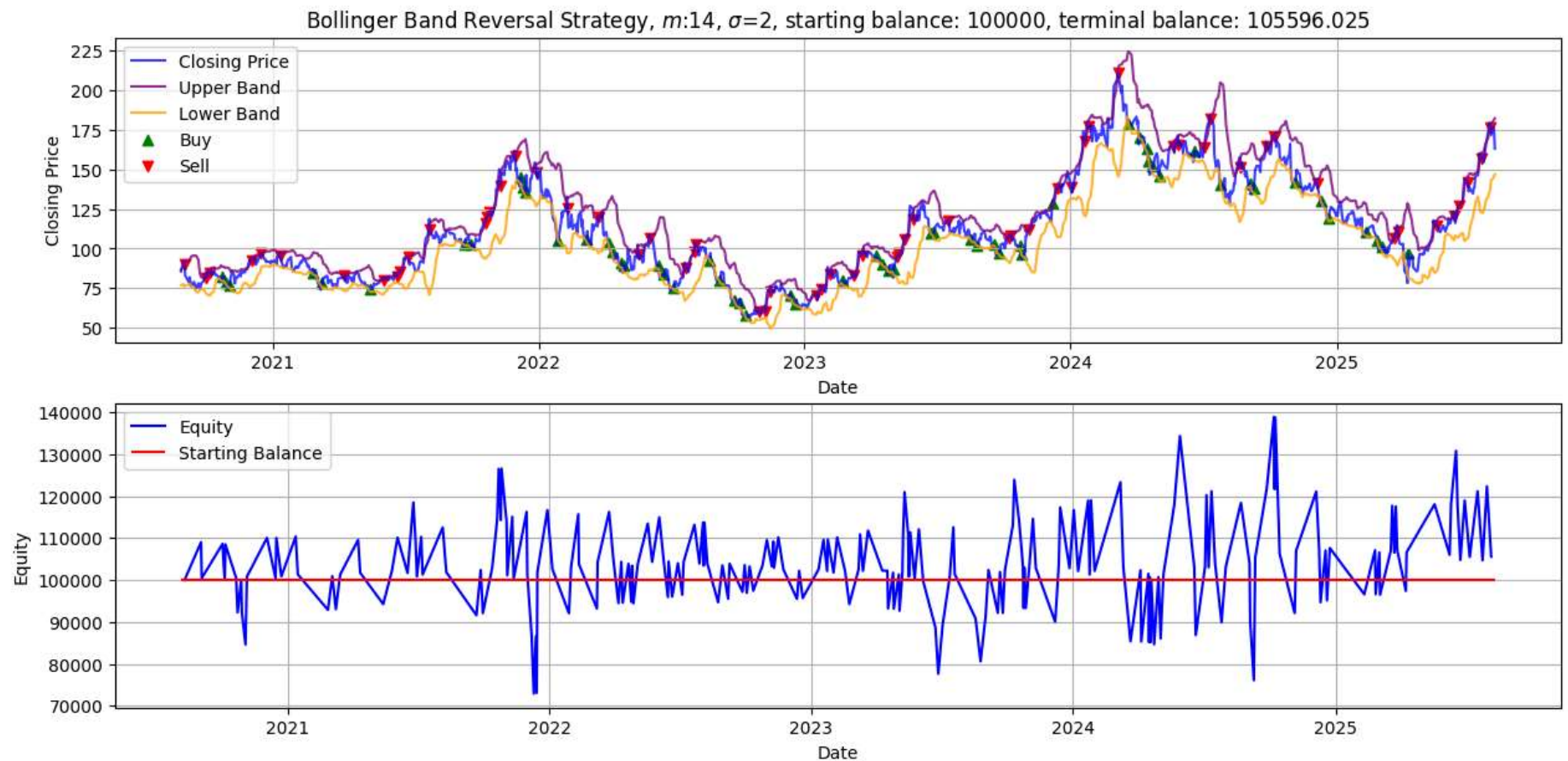
N.b. some closing price data is given as a string (e.g. '\$431.2'). We must convert to a float and remove the '\$'.

```
"""
# import the original dataframe
data = pd.read_csv(data_title)
# change column names:
data = data.rename(columns={date_title: 'Date', closing_price_title: 'Close'})
# we only need the date and closing price columns:
data = data.loc[:, ['Date', 'Close']]
# if dates are given as strings, change to pd.Datetimes
if type(data.iloc[0]['Date'])==str:
    data['Date'] = pd.to_datetime(data['Date'], format='mixed')
# remove '$' from closing price strings if that is how they are given:
a = data.iloc[0]['Close']
if type(a)==str:
    if a[0]=='$':
        data['Close'] = (data['Close'].str[1:]).astype(float)
# if data is reversed, reverse it back:
if data.iloc[0]['Date']>=data.iloc[1]['Date']:
    data = data[::-1]
    data.index = pd.RangeIndex(0, len(data), step=1)
return data
```

```
In [20]: # import the data and use `format` to convert it into the format compatible with our programme:
cny_eur = format("CNY vs EUR.csv", "DATE", 'Chinese yuan renminbi/Euro (EXR.D.CNY.EUR.SP00.A)') # Chinese Yuan vs Euro
usd_eur = format("USDvsEUR.csv", "DATE", "US dollar/Euro (EXR.D.USD.EUR.SP00.A)") # US Dollar vs Euro
qcom = format('QCOM.csv', 'Date', 'Close/Last') # QUALCOMM
meta = format("META.csv", 'Date', 'Close/Last') # META
amd = format("AMD.csv", 'Date', 'Close/Last') # AMD
nvda = format("NVDA.csv", 'Date', 'Close/Last') # NVDA
```

```
In [ ]: # Let's now import some risk free asset data.  
# We use the yield on a 10Yr Tbill:  
risk_free = pd.read_csv("US 10YR Yields.csv")  
risk_free = risk_free.rename(columns={'observation_date': 'Date', 'DGS10': 'Yield'})  
risk_free['Date'] = pd.to_datetime(risk_free['Date'], format="%Y-%m-%d")  
# remove the missing values:  
risk_free = risk_free.dropna()
```

```
In [45]: # now let's carry out the Bollinger band reversal strategy:  
period, width = 14, 2  
starting_balance = 100000  
amd_bollinger_rev = AT.BollingerReversal(starting_balance, amd, period, width, risk_free)  
amd_bollinger_rev.backtest()
```



```
Out[45]: {'Starting Balance': '100000',  
          'PnL': '5596.025',  
          'Return on Equity (%)': '5.596',  
          'Sharpe Ratio': '-0.063',  
          'Volatility (%)': '11.876',  
          'Beta': '0.494'}
```

The metrics printed above tell us that we made a small profit on this strategy, gaining \$5596.03 over roughly 5 years. This is not ideal, but recall we said that Bollinger bands do not perform well on ranging stocks. We see from the above plot that AMD has only a very small upwards trend, but overall looks to be mean-reverting.

Our Sharpe ratio is negative suggesting we were better off investing in the risk-free asset compared to running this trading strategy, and our β is roughly 0.5, suggesting that our trading strategy is about half as reactive to movements in the underlying asset - in this case, the stock itself.

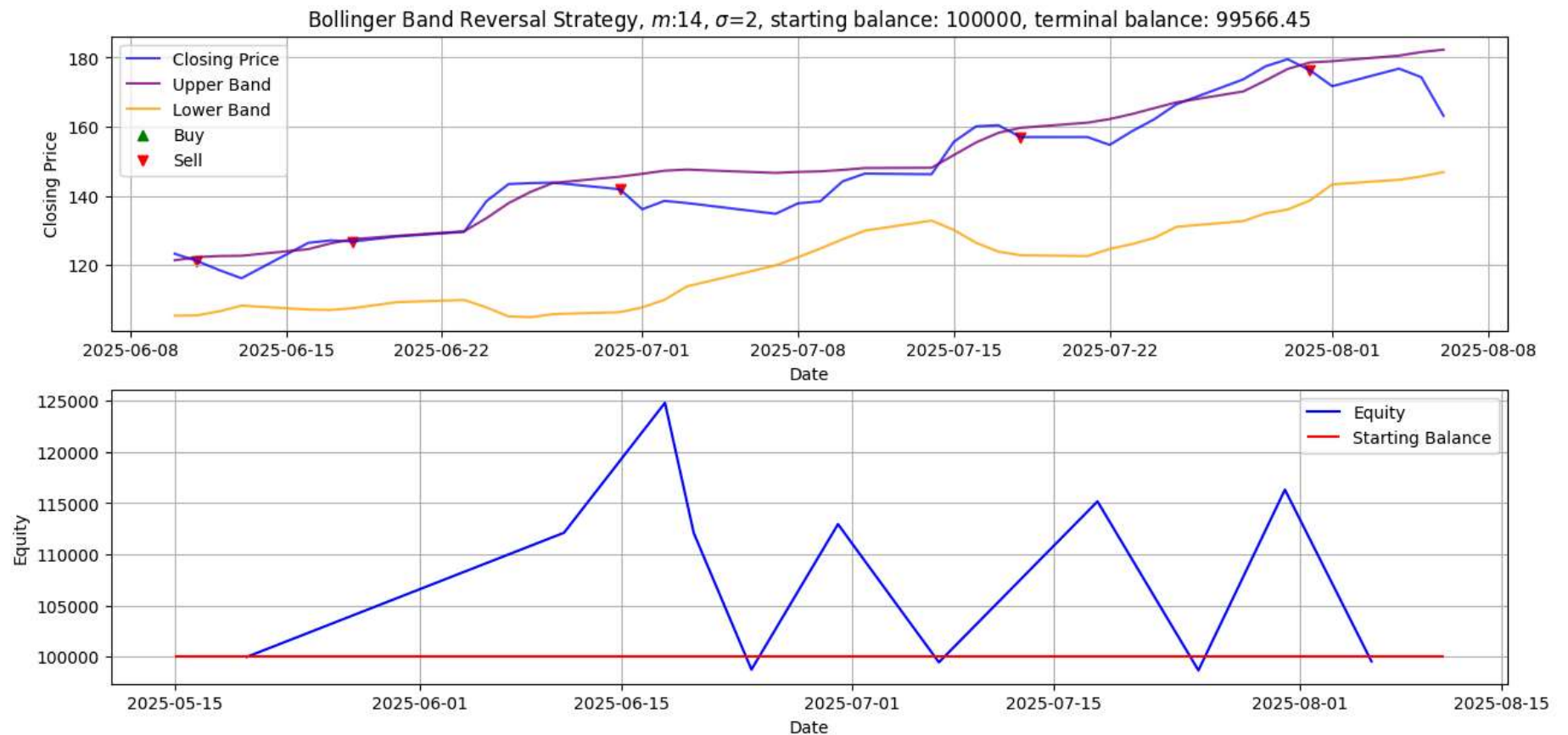
```
In [23]: # Let's examine our orderbook for this strategy:
book = amd_bollinger_rev.book
book.head()
```

```
Out[23]:
```

	Id	Open Date	Open Price	Type	Volume	Stop-Loss	Take-Profit	Status	Close Date	Close Price	PnL
0	20200902100S	2020-09-02	90.22	Sell	100	94.731	85.709	Closed	2020-09-03	85.709	451.1
1	20201002100S	2020-10-02	81.80	Sell	100	85.890	77.710	Closed	2020-10-05	85.890	-409.0
2	20201006100S	2020-10-06	84.48	Sell	100	88.704	80.256	Closed	2020-10-21	80.256	422.4
3	20201023100B	2020-10-23	81.96	Buy	100	77.862	86.058	Closed	2020-10-28	77.862	-409.8
4	20201029100B	2020-10-29	78.02	Buy	100	74.119	81.921	Closed	2020-11-05	81.921	390.1

For each trade, we can identify it via its Id. This way, we can look up any trade that we have made during the backtest. In this particular strategy, we made 121 trades, 58 of which were of `type=="Buy"`, the remaining 63 were `type=="Sell"`. We made a profit on 63 out of 121 trades, with maximum profit being \$1056.9.

```
In [24]: # we do the same strategy, except we now look at smaller subsection of the data:
amd_bollinger_rev = AT.BollingerReversal(starting_balance, amd.iloc[1200:1750], period, width, risk_free)
amd_bollinger_rev.backtest()
```



```
Out[24]: {'Starting Balance': '100000',
          'PnL': '-433.550',
          'Return on Equity (%)': '-0.434',
          'Sharpe Ratio': '0.041',
          'Volatility (%)': '13.570',
          'Beta': '1.338'}
```

Here we end up making a loss of 433.55. Our Sharpe ratio is almost 0, and our β is nearly 1.34. My guess as to why this strategy doesn't work well is because the price is crossing the upper Bollinger band too frequently, on account of being on a clear upwards trend.

MACD - Multi Timeframe

Let's now consider the multi timeframe MACD strategy we mentioned could be done earlier. We take inspiration from an article on medium [10] which did a similar strategy with three simple moving averages (SMAs). In [10], if the same type of crossover happened on all three SMAs, then a trade was placed. Here, we compute the MACD of the given data for three different sets of parameters. We use [5, 13, 5] as the short-term settings, [12, 26, 9] as the standard, medium term settings, and finally [15, 30, 9] as the longer term settings. The idea for these settings came from [7].

The strategy is as follows:

- If an upwards crossover occurs on all three MACDs, we buy.
- If a downwards crossover occurs on all three MACDs, we sell.

We also stipulate the following:

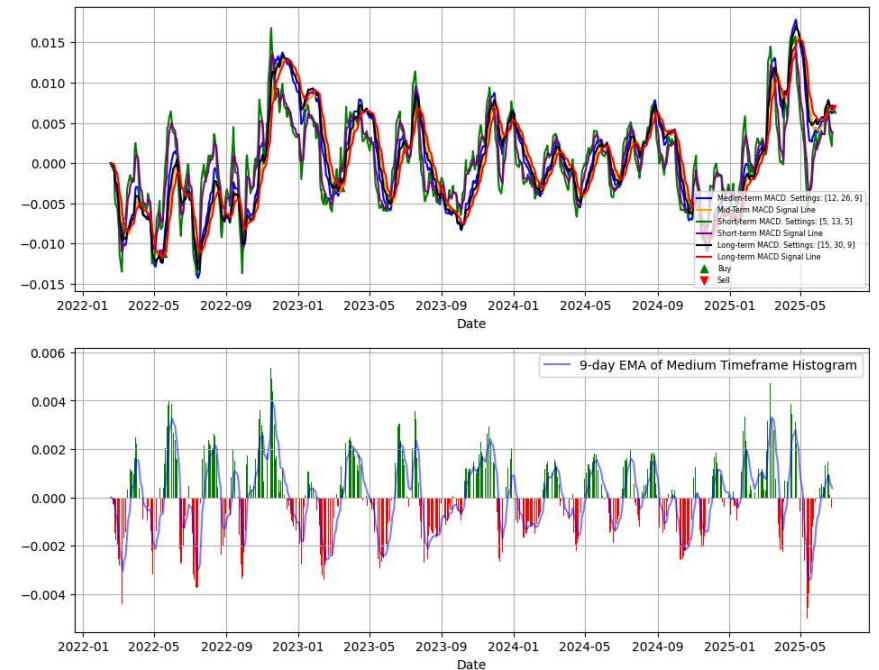
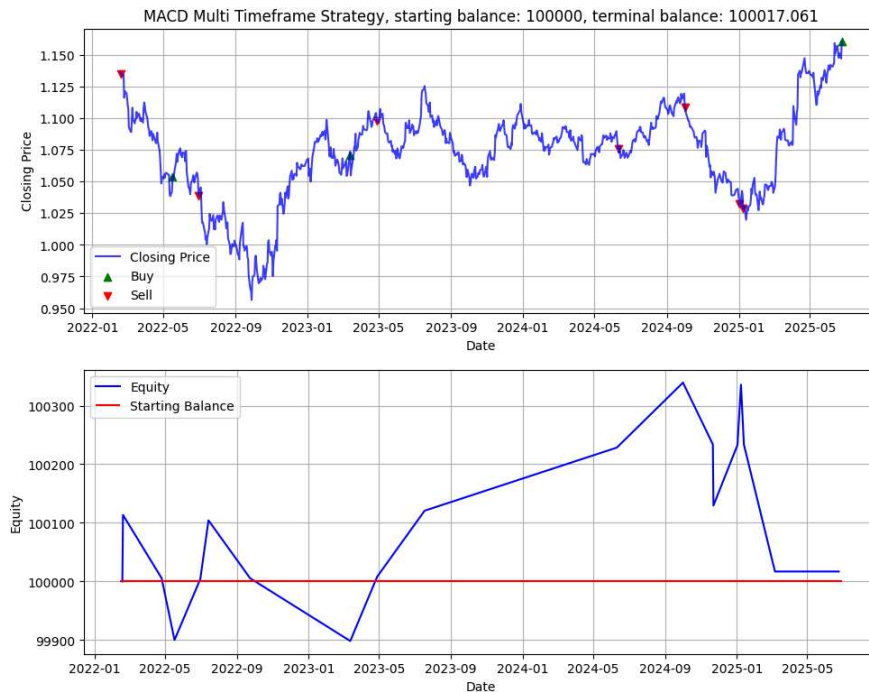
- If an upwards crossover occurs on 2 out of 3 MACDs, and the long-term MACD has just had an 'above zero-line crossover', then we buy.
- If a downwards crossover occurs on 2 out of 3 MACDs, and the long-term MACD has just had a 'below zero-line crossover', then we sell.

To count how many crossovers occur on a given date, we do the following for each MACD:

- Upwards crossover = +1.
- Downwards crossover = -1.

At each date, we have a `Buy Counter` and `Sell Counter`. Then if three upwards crossovers occur, `Buy Counter==+3` for example. If three downwards crossovers occur, `Sell Counter==-3`.

```
In [ ]: # we try the MACD multi timeframe strategy on some FX data taken from the ECB website (see bibliography):
        usd_eur_macd_multitimeframe = AT.MACD_multi_timeframe(starting_balance, usd_eur.iloc[7*len(usd_eur)//8:], [5, 13, 5], [12, 26, 9], [15, 30, 9])
        usd_eur_macd_multitimeframe.backtest()
```



```
Out[ ]: {'Starting Balance': '100000',
         'PnL': '17.061',
         'Return on Equity (%)': '0.017',
         'Sharpe Ratio': '-0.424',
         'Volatility (%)': '0.112',
         'Beta': '-0.003'}
```

Once again the margins are slim. I have a feeling this has been exacerbated by the unsystematically chosen parameter settings for the MACD, and the randomly chosen stop-loss and take-profit thresholds for each trade, which have been kept at 5% for all strategies in the programme.

Moreover, the plot of all three MACD lines has no use to us since it is far too difficult to read from. It looks more like a piece of graffiti one see's by the side of a train track. The equity curve shows that we should've terminated the trading session around October 2024 to achieve the maximum profit, although even then the margins are of no significance.

As I mentioned at the beginning, the purpose of this project was not to show that I know lots about algorithmic trading strategies. What I wanted to do here was learn a bit about some key technical indicators used often by traders, learn about how one goes about developing an algorithmic trading strategy, and show how we can backtest strategies on historical data using Python.

The final strategy we will look at is a combination of the MACD and the RSI.

MACD and RSI Combined

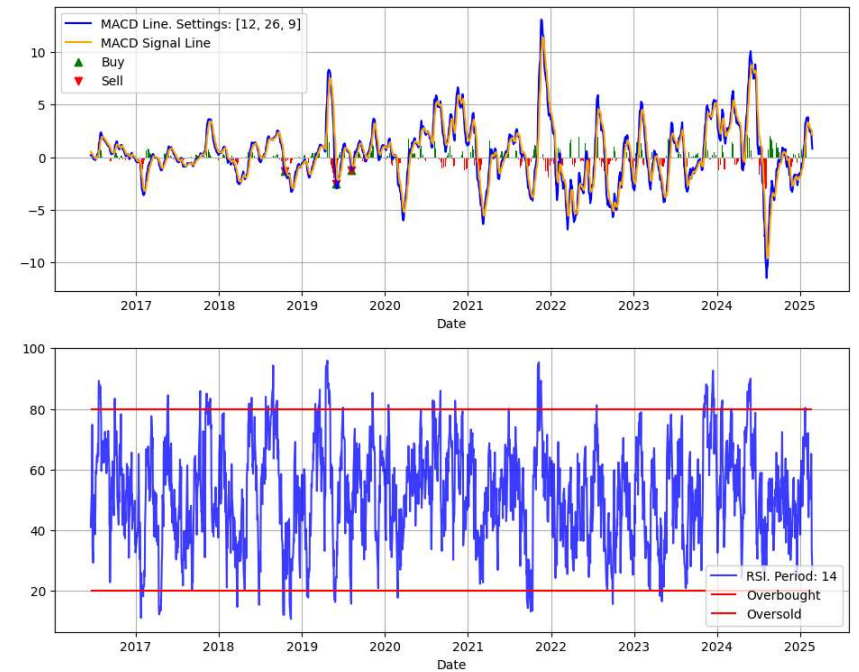
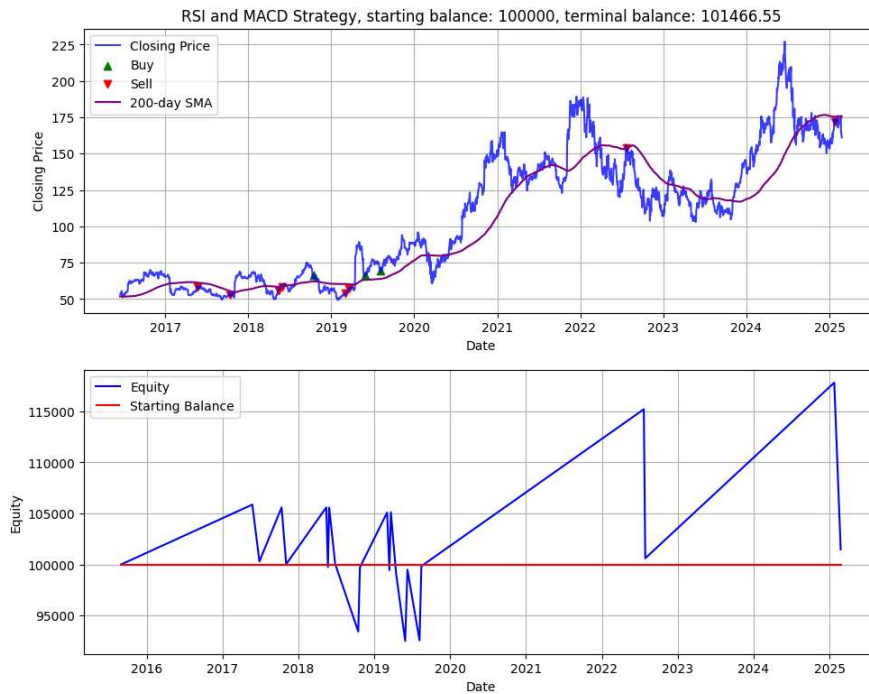
Here we define a trading strategy in which we use the standard RSI reversal strategy in conjunction with the MACD. We first establish if there exists an upwards or downwards trend using the MACD and a simple moving average (SMA) with a long-term period, e.g. 100 days.

- If the price is acting above the SMA or if the MACD line has crossed above 0, we are in an upwards trend.
- If the price is below the SMA, or if the MACD has crossed below 0, then we are in a downwards trend.

Then:

- Whilst in an upwards trend, if the RSI crosses above the 'Oversold' line after previously falling below it, we buy.
- Whilst in a downwards trend, if the RSI falls below the 'Overbought' line after previously rising above it, we sell.

```
In [38]: qcom_macd_rsi = AT.RSI_and_MACD(starting_balance, qcom, [12, 26, 9], 14, 80, 20, 200, risk_free)
qcom_macd_rsi.backtest()
```



```
Out[38]: {'Starting Balance': '100000',
          'PnL': '1466.550',
          'Return on Equity (%)': '1.467',
          'Sharpe Ratio': '-0.186',
          'Volatility (%)': '8.379',
          'Beta': '0.168'}
```

```
In [48]: qcom_macd_rsi.book.head()
```

Out[48]:

	Id	Open Date	Open Price	Type	Volume	Stop-Loss	Take-Profit	Status	Close Date	Close Price	PnL
0	20170524100S	2017-05-24	58.64	Sell	100	61.5720	55.7080	Closed	2017-06-27	55.7080	293.20
1	20171012100S	2017-10-12	53.00	Sell	100	55.6500	50.3500	Closed	2017-11-03	55.6500	-265.00
2	20180515100S	2018-05-15	55.64	Sell	100	58.4220	52.8580	Closed	2018-05-23	58.4220	-278.20
3	20180529100S	2018-05-29	58.23	Sell	100	61.1415	55.3185	Closed	2018-06-27	55.3185	291.15
4	20181016100B	2018-10-16	66.12	Buy	100	62.8140	69.4260	Closed	2018-10-24	62.8140	-330.60

Backtesting is not the be-all and end-all of how traders test a strategy. It should not be over-relied on. For instance, backtesting considers only **historical data**. Whilst it's true that some assets are considered to be mean-reverting, and thus may exhibit similar behaviour over time, we cannot guarantee that this behaviour will occur when we want it to. Backtesting does not take into consideration the market conditions during the time horizon of the backtest. This brings us to an important point that we learn from [11]:

The backward-looking nature of backtesting can be taken advantage of to our benefit as well as our detriment. If we consider the current market to display certain traits which are similar to a given time period in the past, we can test on this particular time period to examine how the trading strategy behaves during those particular conditions. This of course relies on us being right about the future characteristics of our market. However, in doing this, we are exposing ourselves to 'backtesting bias', where we examine the performance of a strategy on data with characteristics which we have selected beforehand. As stated in [11], we must backtest our strategy on many different datasets over many different time intervals to gain an objective set of results.

There are other biases that we must be careful not to fall foul to. See [11] for further details.

Next Steps:

- Determine optimal stop-loss and take-profit thresholds.
- Optimise technical indicator hyperparameters - e.g. MACD settings or RSI overbought and oversold levels.
- Study examples of trading strategies in depth and learn about on which asset classes they are best deployed.

Sources:

- [1] ATJ Traders - Algo Trading. Relevant YouTube Video: <https://www.youtube.com/watch?v=I5unWZBldus&t=309s>.
- [2] Ch 11.1 of Tze Leung Lai, Haipeng Xing, Statistical Models and Methods for Financial Markets, Springer New York, NY, 2008.
- [3] <https://corporatefinanceinstitute.com/resources/equities/bollinger-bands/> .
- [4] <https://corporatefinanceinstitute.com/resources/career-map/sell-side/capital-markets/exponential-moving-average-ema/> (for EMAs)
- [5] <https://www.oanda.com/us-en/learn/indicators-oscillators/determining-entry-and-exit-points-with-macd/> (for the MACD)
- [6] <https://www.oanda.com/us-en/trade-tap-blog/trading-knowledge/understanding-the-relative-strength-index/>
- [7] <https://www.mindmathmoney.com/articles/the-complete-macd-indicator-trading-guide-master-price-momentum-in-2025>
- [8] <https://www.oanda.com/us-en/trade-tap-blog/analysis/technical/mastering-rsi-trading-strategies/>
- [9] <https://hmarkets.com/uk/learn-to-trade/learning-hub/bollinger-bands/#:~:text=Trading%20breakout%20strategy%20using%20Bollinger%20bands&text=The%20basic%20idea%20behind%20the,breaks%20>
- [10] https://medium.com/@redsword_23261/multi-timeframe-macd-indicator-crossover-trading-strategy-c6821366a1fe
- [11] <https://corporatefinanceinstitute.com/resources/data-science/backtesting/>
- [12] <https://www.quantstart.com/articles/Sharpe-Ratio-for-Algorithmic-Trading-Performance-Measurement/>
- [13] <https://www.cmcmarkets.com/en-gb/trading-guides/stock-beta>

The historical data used in this project are taken from:

- <https://www.nasdaq.com/market-activity/quotes/historical> (for equity data)
- <https://www.ecb.europa.eu/home/html/index.en.html> (for currency data)
- Board of Governors of the Federal Reserve System (US), Market Yield on U.S. Treasury Securities at 10-Year Constant Maturity, Quoted on an Investment Basis [DGS10], retrieved from FRED, Federal Reserve Bank of St. Louis; <https://fred.stlouisfed.org/series/DGS10>, September 2, 2025. (For risk free yield data)